LifeOS: The Personal Chaos Agent - Detailed Project Explanation

1. Project Overview: What is LifeOS?

LifeOS is a **Reinforcement Learning (RL) environment** designed to train Large Language Models (LLMs) to handle the chaotic, cascading conflicts of real personal life.

The Problem: Current LLMs are great at structured tasks (coding, math) but struggle with personal life management where decisions have compounding consequences (e.g., choosing between finishing an assignment, resting to avoid burnout, or replying to an angry friend).
The Solution: LifeOS simulates a "chaotic student week". The LLM agent acts inside this simulation, making step-by-step decisions. Based on how well it manages tasks, relationships, and its own energy, it receives rewards or penalties. Over many episodes, the LLM learns optimal strategies to survive the chaos.

---

2. Core Architecture: How does it work?

The project follows the **OpenEnv standard**, which strictly separates the environment (the world rules) from the agent (the LLM making decisions).

- **Environment (`lifeos/envs/student_week_openenv.py`)**: The core engine. It maintains the state of the world (energy, tasks, calendar) and defines how actions change that state.
- **Server (`lifeos/envs/server.py`)**: A FastAPI application that wraps the environment, exposing it via HTTP endpoints (`/env/reset`, `/env/step`).
- **Client (`lifeos/envs/client.py`)**: A lightweight Python HTTP client used to interact with the server.
- **Reward Modules (`lifeos/rewards/`)**: Independent scripts that calculate scores based on the agent's actions.
- **Training Script (`lifeos/training/train_grpo.py`)**: The script that actually trains the LLM using the environment's feedback.
- **User Interface (`web_app.py` / `spaces/app.py`)**: A Gradio web dashboard for humans to watch the agent play out a scenario.

---

3. Deep Dive: The Environment

The environment is a turn-based simulation. Each "episode" lasts for a maximum of 30 steps.

What the Agent Sees (Observation Space)

At every step, the agent receives a snapshot of its life:

- **Stats**: Energy (0-100), Stress (0-100), Budget (₹), Relationship Score (0.0 - 1.0).
- **Lists**: Pending Tasks (with deadlines and effort), Inbox (messages needing replies), Calendar (upcoming events and conflicts).

- **Chaos Events**: Random surprises (e.g., "Assignment moved up!") that trigger dynamically.

What the Agent Can Do (Action Space)

The agent must choose one of 6 structured actions per step:

1. `prioritize_task(target_id, urgency_level)`: Spend effort on a task.
2. `reply_message(target_id, tone, content_summary)`: Answer a text/email.
3. `reschedule_event(target_id, new_time, reason)`: Move a calendar block.
4. `delegate_task(target_id, reason)`: Pay money (from budget) to get a task done.
5. `decline_event(target_id, reason)`: Cancel an event (lowers relationship score).
6. `rest()`: Do nothing, recover energy, reduce stress.

The Game Loop (Step Function)

When the agent submits an action:

1. **Format Check**: Is the action valid JSON and compliant with the schema?
2. **Anti-Hack Check**: Is the agent repeating the same action infinitely?
3. **Execution**: Apply the action (e.g., reduce task effort, mark message as replied).
4. **World Update**: Decrease energy naturally, increase stress, advance time, deliver new messages.
5. **Chaos Injection**: 20% chance a random bad/good thing happens.
6. **Reward Calculation**: Compute the score for this specific step.

---

4. The Reward System (How the Agent Learns)

To prevent "reward hacking" (where the AI finds a cheat to get a high score without doing the work), LifeOS uses **4 independent reward functions**:

1. **Task Completion (`task_completion_reward.py`)**:
    - +1.0 for finishing a task before the deadline.
    - -1.0 for missing a deadline.
    - -0.5 for unnecessarily delegating a low-priority task.

2. **Social Coherence (`social_coherence_reward.py`)**:
    - +0.5 for replying to a message quickly (within 1 step).
    - -0.8 for leaving messages unread by the end of the episode.

3. **Energy Sustainability (`energy_sustainability_reward.py`)**:
    - +0.2 for every step energy stays above 40.
    - +0.4 for taking a proactive `rest()` before energy gets dangerously low.
    - **-1.5 (and instant Game Over)** if energy hits 0 (Burnout).

4. **Format Compliance (`format_compliance_reward.py`)**:
    - +0.1 for outputting perfect, valid action commands.

- -2.0 if the LLM takes too long (timeout).

---

5. The Training Pipeline (`train_grpo.py`)

This is where the actual Machine Learning happens. We use **GRPO (Group Relative Policy Optimization)**.

1. **The Setup**: We use a base LLM (`Mistral-7B-Instruct-v0.3`) and load it efficiently using `Unsloth` (4-bit quantization, so it fits on consumer GPUs).
2. **The Loop**:
    - The environment `reset()`s.
    - The LLM looks at the Observation and generates a text response (the Action).
    - The environment `step()`s the action and returns a Reward.
    - GRPO takes these rewards and updates the LLM's weights (via a LoRA adapter) so it is more likely to choose high-reward actions next time.
3. **The Result**: Over 50 episodes, the LLM stops making silly mistakes (like working until burnout) and learns to balance resting, replying to friends, and hitting deadlines.

---

6. Deployment (Hugging Face Spaces)

The project is packaged for the **Meta OpenEnv Hackathon** and deployed on Hugging Face Spaces.

- `Dockerfile.openenv`: Tells Hugging Face how to install dependencies and run the server.
- `openenv.yaml`: The official manifest file declaring the environment's capabilities to the OpenEnv framework.
- `web_app.py`: A Gradio interface that connects to the environment, allowing judges to click "Run Episode" and visually see the LLM's decisions step-by-step.

Summary

LifeOS takes the messy reality of human life—limited energy, conflicting schedules, and unexpected chaos—and turns it into a mathematical game. By forcing an LLM to play this game thousands of times using Reinforcement Learning, we teach it to become a highly effective, context-aware personal assistant.